


# CSEB3313 Object-Oriented Programming

**PUTRAJAYA CAMPUS**

**OBJECT-ORIENTED PROGRAMMING (CSEB3313)**

**SECTION 02**

**SEMESTER 2024/2025**

**GROUP ASSIGNMENT**

**LECTURER:**

**MOAMIN A MAHMOUD, TS. DR.**

Student 1: MUHAMMAD IRFAN BIN ISMAIL ID: SN01083706 Section 02B

Student 2: SELVAPRADNESH A/L SELVASURESH ID: SW01083419 Section 02A

Student 3: AHMAD FIKRI BIN AHMAD MUSTAPHA ID: CS01083393 Section 02B

Student 4: MOHAMMAD FAYYADH BIN MOHAMMAD FAKRY ID: CS01083682 Section 02B

Student 5: DANISH IQBAL BIN NORHASBI ID: SW01083800 Section 02B

# Introduction

The following report presents the source code to design a Java-based banking System for ABC BANK using Object-Oriented Programming (OOP) principles. The proposed design for the system focused on the implementation of some key features such as file processing, exception handling, data validation, inheritance, graphical user interface usage , and report generation.

The source code is provided in the report to show in detail what steps were taken to solve the problems of the needed banking system. Comments can be found to help guide the reader through the complex lines of code. Screenshots of the code and its output results are also included in this report to provide evidence on the effectiveness of the code's design overall.

Finally, a conceptual description of 4 key Object-Oriented Programming principles explains how we implemented the knowledge learnt from these chapters into our coding to help improve its efficiency and appeal.

Full Source Code

```
import java.util.*;
import javax.swing.*;
import java.text.SimpleDateFormat;
import java.io.FileWriter;
import java.io.IOException;

class Transaction {
    String type;
    double amount;
    String date;

    public Transaction(String type, double amount) {
        this.type = type;
        this.amount = amount;
        this.date = new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss").format(new Date());
    }

    @Override
    public String toString() {
        return "Type: " + type + ", Amount: " + amount + ", Date: " +
date;
    }
}

class BankAccount {
    private String accountID;
    private String customerID;
```

```
private ArrayList<Transaction> transactionHistory;

public BankAccount() {
    this.accountID = "Empty";
    this.customerID = "Empty";
    this.transactionHistory = new ArrayList<>();
}

public BankAccount(String accountID, String customerID) {
    this.accountID = accountID;
    this.customerID = customerID;
    this.transactionHistory = new ArrayList<>();
}

public String getAccID() {
    return accountID;
}

public void setAccID(String accountID) {
    this.accountID = accountID;
}

public String getCustomerID() {
    return customerID;
}

public void setCustomerID(String customerID) {
    this.customerID = customerID;
}
```

```
    public void addTransaction(String type, double amount) {
        Transaction transaction = new Transaction(type, amount);
        transactionHistory.add(transaction);
        logTransaction(transaction);
    }

    private void logTransaction(Transaction transaction) {
        try (FileWriter writer = new FileWriter("transactions.log",
true)) {
            String timestamp = new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss").format(new Date());
            writer.write(timestamp + " | " + this + " | " + transaction +
"\n");
        } catch (IOException e) {
            JOptionPane.showMessageDialog(null, "Failed to log
transaction: " + e.getMessage());
        }
    }

    public ArrayList<Transaction> getTransactionHistory() {
        return transactionHistory;
    }

    @Override
    public String toString() {
        return "AccountID = " + accountID + ", CustomerID = " +
customerID;
    }

    public void viewTransactionHistory() {
```

```
        if (transactionHistory.isEmpty()) {
            JOptionPane.showMessageDialog(null, "No transactions
found.");
            return;
        }
        StringBuilder history = new StringBuilder("Transaction
History:\n");
        for (Transaction transaction : transactionHistory) {
            history.append(transaction).append("\n");
        }
        JOptionPane.showMessageDialog(null, history.toString());
    }
}

class SavingAccount extends BankAccount {
    private double saveBal;

    public SavingAccount(String accountID, String customerID,
double saveBal) {
        super(accountID, customerID);
        this.saveBal = saveBal;
    }

    public double getSaveBal() {
        return saveBal;
    }

    public void setSaveBal(double saveBal) {
        this.saveBal = saveBal;
    }
```

```
    @Override
    public void addTransaction(String type, double amount) {
        super.addTransaction(type, amount);
        if (type.equalsIgnoreCase("Deposit")) {
            saveBal += amount;
        } else if (type.equalsIgnoreCase("Withdraw")) {
            saveBal -= amount;
        }
    }

    @Override
    public String toString() {
        return super.toString() + ", Saving Balance: " + saveBal;
    }
}

class CurrentAccount extends BankAccount {
    private double currentBal;

    public CurrentAccount(String accountID, String customerID,
double currentBal) {
        super(accountID, customerID);
        this.currentBal = currentBal;
    }

    public double getCurrentBal() {
        return currentBal;
    }
```

```
    public void setCurrentBal(double currentBal) {
        this.currentBal = currentBal;
    }

    @Override
    public void addTransaction(String type, double amount) {
        super.addTransaction(type, amount);
        if (type.equalsIgnoreCase("Deposit")) {
            currentBal += amount;
        } else if (type.equalsIgnoreCase("Withdraw")) {
            currentBal -= amount;
        }
    }

    @Override
    public String toString() {
        return super.toString() + ", Current Balance: " + currentBal;
    }
}

public class Menu {
    private static final BankAccount[] accountArray = new
BankAccount[100];
    private static int accountCount = 0;

    public static void main(String[] args) {
        boolean exit = false;

        while (!exit) {
            String menu = "Bank Account Management System:\n" +
```

```
                "1. Create Account\n" +
                "2. View Account Details\n" +
                "3. Deposit/Withdraw\n" +
                "4. Transfer Funds\n" +
                "5. View Transaction History\n" +
                "6. Generate and View Reports\n" +
                "7. Exit";

        String choice = JOptionPane.showInputDialog(menu);
        if (choice == null) continue; // Handle cancel case

        try {
            switch (choice) {
                case "1": {
                    String accountType =
JOptionPane.showInputDialog("Select account type
(Savings/Current):");

                    if (!"Savings".equalsIgnoreCase(accountType)
&& !"Current".equalsIgnoreCase(accountType)) {
                        JOptionPane.showMessageDialog(null, "Invalid
account type. Please select Savings or Current.");
                        break;
                    }

                    String accountID =
JOptionPane.showInputDialog("Enter your new account ID:");
                    String customerID =
JOptionPane.showInputDialog("Enter your customer ID:");
```

```
                    double balance =
Double.parseDouble(JOptionPane.showInputDialog("Enter your
initial balance:"));

                    BankAccount account;
                    if ("Savings".equalsIgnoreCase(accountType)) {
                        account = new SavingAccount(accountID,
customerID, balance);
                    } else {
                        account = new CurrentAccount(accountID,
customerID, balance);
                    }

                    accountArray[accountCount++] = account;
                    logAccountCreation(account); // Log account
creation with timestamp
                    JOptionPane.showMessageDialog(null, "Account
created successfully!\n" + account);
                    break;
                }
                case "2": {
                    String accountID =
JOptionPane.showInputDialog("Enter your account ID:");
                    BankAccount account =
getAccountById(accountID);
                    if (account != null) {
                        JOptionPane.showMessageDialog(null,
account);
                    } else {
```

```
                    JOptionPane.showMessageDialog(null,
"Account not found.");
                }
                break;
            }
            case "3": {
                String accountID =
JOptionPane.showInputDialog("Enter your account ID:");
                BankAccount account =
getAccountById(accountID);
                if (account == null) {
                    JOptionPane.showMessageDialog(null,
"Account not found.");
                    break;
                }

                String transactionType =
JOptionPane.showInputDialog("Enter transaction type
(Deposit/Withdraw):");
                double amount =
Double.parseDouble(JOptionPane.showInputDialog("Enter
amount:"));

                if ("Deposit".equalsIgnoreCase(transactionType) ||
"Withdraw".equalsIgnoreCase(transactionType)) {
                    account.addTransaction(transactionType,
amount);
                    JOptionPane.showMessageDialog(null,
transactionType + " successful.");
                } else {
```

```
                    JOptionPane.showMessageDialog(null, "Invalid
transaction type.");
                }
                break;
            }
            case "4": {
                String sourceAccountID =
JOptionPane.showInputDialog("Enter your source account ID:");
                BankAccount sourceAccount =
getAccountById(sourceAccountID);
                if (sourceAccount == null) {
                    JOptionPane.showMessageDialog(null, "Source
account not found.");
                    break;
                }

                String destinationAccountID =
JOptionPane.showInputDialog("Enter destination account ID:");
                BankAccount destinationAccount =
getAccountById(destinationAccountID);
                if (destinationAccount == null) {
                    JOptionPane.showMessageDialog(null,
"Destination account not found.");
                    break;
                }

                double amount =
Double.parseDouble(JOptionPane.showInputDialog("Enter
amount to transfer:"));
```

```
                    sourceAccount.addTransaction("Withdraw",
amount);
                    destinationAccount.addTransaction("Deposit",
amount);
                    JOptionPane.showMessageDialog(null, "Transfer
successful.");
                    break;
                }
                case "5": {
                    String accountID =
JOptionPane.showInputDialog("Enter your account ID:");
                    BankAccount account =
getAccountById(accountID);
                    if (account != null) {
                        account.viewTransactionHistory();
                    } else {
                        JOptionPane.showMessageDialog(null,
"Account not found.");
                    }
                    break;
                }
                case "6": {
                    StringBuilder report = new StringBuilder("Bank
Account Report:\n");
                    String timestamp = new SimpleDateFormat("yyyy-
MM-dd HH:mm:ss").format(new Date());
                    report.append("Generated on:
").append(timestamp).append("\n");

                    for (int i = 0; i < accountCount; i++) {
```

```
                    BankAccount account = accountArray[i];
                    report.append(account).append("\n");
                    for (Transaction transaction :
account.getTransactionHistory()) {
                        report.append("
").append(transaction).append("\n");
                    }
                }

                saveReportToFile(report.toString());
                JOptionPane.showMessageDialog(null, "Report
saved to summary_report.txt");
                break;
            }
            case "7": {
                exit = true;
                JOptionPane.showMessageDialog(null, "Exiting
the system.");
                break;
            }
            default: {
                JOptionPane.showMessageDialog(null, "Invalid
choice. Please try again.");
            }
        }
    } catch (Exception e) {
        JOptionPane.showMessageDialog(null, "Error: " +
e.getMessage());
    }
}
```

```
    }

    private static BankAccount getAccountById(String accountID) {
        for (int i = 0; i < accountCount; i++) {
            if (accountArray[i].getAccID().equals(accountID)) {
                return accountArray[i];
            }
        }
        return null;
    }

    private static void logAccountCreation(BankAccount account) {
        try (FileWriter writer = new FileWriter("account_creation.log",
true)) {
            String timestamp = new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss").format(new Date());
            writer.write(timestamp + " | " + account + "\n");
        } catch (IOException e) {
            JOptionPane.showMessageDialog(null, "Failed to log
account creation: " + e.getMessage());
        }
    }

    private static void saveReportToFile(String report) {
        try (FileWriter writer = new FileWriter("summary_report.txt"))
{
            writer.write(report);
        } catch (IOException e) {
            JOptionPane.showMessageDialog(null, "Failed to save
report: " + e.getMessage());
```

```
        }
    }
}
```

**1. Object-Oriented Programming:**

```
class Transaction {
// Class to represent a transaction (Deposit or Withdraw)

    String type;
// Type of transaction (Deposit or Withdraw)

    double amount;
// Amount of money involved in the transaction

    String date;  // Date and time of the transaction

        public Transaction(String type, double amount) {
// Constructor to initialize the transaction with type and
amount

        this.type = type;  // Set the type of transaction

        this.amount = amount;  // Set the amount involved
```

```
        this.date = new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss").format(new Date());
// Format the current date and time for the transaction
    }


    @Override

    public String toString() {
        return "Type: " + type + ", Amount: " + amount + ", Date:
" + date;
// Override toString to return a the transaction in string
    }
}


class BankAccount {
// Class to represent a bank account

    private String accountID;
// Account ID for the bank account

    private String customerID;
// Customer ID for the account holder

    private ArrayList<Transaction> transactionHistory;
// List to store all transactions for this account

    public BankAccount() {
```

```
// Default constructor to initialize the bank account with
"Empty" values

        this.accountID = "Empty";  // Set account ID to "Empty"
        this.customerID = "Empty";
  // Set customer ID to "Empty"

        this.transactionHistory = new ArrayList<>();
// Initialize an empty transaction history list
    }


 public BankAccount(String accountID, String customerID) {
// Constructor to initialize the bank account with provided
account and customer IDs

        this.accountID = accountID;  // Set the account ID

        this.customerID = customerID;  // Set the customer ID

        this.transactionHistory = new ArrayList<>();
// Initialize an empty transaction history list
    }


    public String getAccID() {  // Getter method for account ID

        return accountID;  // Return the account ID
    }
```

```
    public void setAccID(String accountID) {
// Setter method for account ID

        this.accountID = accountID;  // Set the account ID
    }


    public String getCustomerID() {// Getter method for
customer ID
        return customerID;  // Return the customer ID
    }


    public void setCustomerID(String customerID) { // Setter
method for customer ID
        this.customerID = customerID;  // Set the customer ID
    }


    public void addTransaction(String type, double amount) {
// Method to add a transaction (Deposit or Withdraw) to the
transaction history

        Transaction transaction = new Transaction(type,
amount);
// Create a new transaction object with the given type and
amount

        transactionHistory.add(transaction);
```

```
 // Add the transaction to the transaction history list

        logTransaction(transaction);
// Log the transaction to a file
    }


    private void logTransaction(Transaction transaction) {
// Method to log the transaction into a file

        try (FileWriter writer = new FileWriter("transactions.log",
true)) {  // Open a file writer to append to the log file
            String timestamp = new SimpleDateFormat("yyyy-
MM-dd HH:mm:ss").format(new Date());  // Get the current
timestamp
            // Write the log entry with timestamp, account info,
and transaction info
            writer.write(timestamp + " | " + this + " | " +
transaction + "\n");
        } catch (IOException e) {  // Catch any IO exceptions
that may occur
            // Show an error message if logging fails
            JOptionPane.showMessageDialog(null, "Failed to log
transaction: " + e.getMessage());
        }
    }

    // Getter method to retrieve the transaction history
    public ArrayList<Transaction> getTransactionHistory() {
```

```
        return transactionHistory;  // Return the list of
transactions
    }

    // Override toString method to return a string
representation of the bank account
    @Override
    public String toString() {
        return "AccountID = " + accountID + ", CustomerID = " +
customerID;
    }

    // Method to view the transaction history in a readable
format
    public void viewTransactionHistory() {
        if (transactionHistory.isEmpty()) {  // Check if there are
no transactions
            // Show a message saying no transactions are found
            JOptionPane.showMessageDialog(null, "No
transactions found.");
            return;  // Exit the method
        }
        // StringBuilder to accumulate the history text
        StringBuilder history = new StringBuilder("Transaction
History:\n");
        // Loop through each transaction in the transaction
history
        for (Transaction transaction : transactionHistory) {
            // Append each transaction's details to the history
string
```

```
            history.append(transaction).append("\n");
        }
        // Show the full transaction history in a dialog
        JOptionPane.showMessageDialog(null,
history.toString());
    }
}
```

**2. Exception Handling:**

```
// Method to log a transaction into a log file
private void logTransaction(Transaction transaction) {
    try (FileWriter writer = new FileWriter("transactions.log",
true)) {  // Open a file writer to append to the
"transactions.log" file
        // Get the current timestamp in the format "yyyy-MM-dd
HH:mm:ss"
        String timestamp = new SimpleDateFormat("yyyy-MM-
dd HH:mm:ss").format(new Date());
        // Write the log entry with timestamp, account details,
and transaction details
        writer.write(timestamp + " | " + this + " | " + transaction +
"\n");
    } catch (IOException e) {  // Catch any IOExceptions that
may occur while writing to the file
        // If there's an error writing to the file, show a message
box with the error details
```

```
        JOptionPane.showMessageDialog(null, "Failed to log
transaction: " + e.getMessage());
    }
}

// Static method to log account creation into a log file
private static void logAccountCreation(BankAccount
account) {
    try (FileWriter writer = new
FileWriter("account_creation.log", true)) {  // Open a file
writer to append to the "account_creation.log" file
        // Get the current timestamp in the format "yyyy-MM-dd
HH:mm:ss"
        String timestamp = new SimpleDateFormat("yyyy-MM-
dd HH:mm:ss").format(new Date());
        // Write the log entry with timestamp and account
details
        writer.write(timestamp + " | " + account + "\n");
    } catch (IOException e) {  // Catch any IOExceptions that
may occur while writing to the file
        // If there's an error writing to the file, show a message
box with the error details
        JOptionPane.showMessageDialog(null, "Failed to log
account creation: " + e.getMessage());
    }
}

// Static method to save a report to a text file
private static void saveReportToFile(String report) {
```

```
    try (FileWriter writer = new
FileWriter("summary_report.txt")) {  // Open a file writer to
overwrite the "summary_report.txt" file
        // Write the provided report content to the file
        writer.write(report);
    } catch (IOException e) {  // Catch any IOExceptions that
may occur while writing to the file
        // If there's an error writing to the file, show a message
box with the error details
        JOptionPane.showMessageDialog(null, "Failed to save
report: " + e.getMessage());
    }
}
```

**3. Assertions:**

```
assert accountID != null : "Account ID cannot be null";
// Ensure that the ‘accountID’ variable is not null.
// If it is null, the assertion will fail and the program will throw
an ‘AssertionError’ with the message "Account ID cannot be
null".

assert ("Savings".equalsIgnoreCase(accountType)||
"Current".equalsIgnoreCase(accountType)): “Account type is
invalid”;
```

*// Ensure that the ‘accountType’ variable is either Savings or Current regardless of case.*
*// If it is not, the assertion will fail and the program will throw an ‘AssertionError’ with the message "Account type is invalid”.*

assert amount > 0 : "Amount must be greater than zero";
*// Ensure that the ‘amount’ variable is greater than zero.*
*// If the value of ‘amount’ is zero or negative, the assertion will fail and the program will throw an ‘AssertionError’ with the message "Amount must be greater than zero".*

**4. Arrays:**

// Declare a constant array to store up to 100 BankAccount objects
private static final BankAccount[] accountArray = new BankAccount[100];  // Fixed-size array to hold up to 100 BankAccount objects

*// Declare and initialize a variable to keep track of the number of accounts created*
private static int accountCount = 0;  // Tracks the number of bank accounts in use (starts at 0)

**5. Loops and Control Flow:**

```
// Loop until the user chooses to exit the program
while (!exit) {
    // Define the menu options as a string with line breaks
    String menu = "Bank Account Management System:\n" +
            "1. Create Account\n" +
            "2. View Account Details\n" +
            "3. Deposit/Withdraw\n" +
            "4. Transfer Funds\n" +
            "5. View Transaction History\n" +
            "6. Generate and View Reports\n" +
            "7. Exit";

    // Show the menu and get the user's choice as input
    String choice = JOptionPane.showInputDialog(menu);
    if (choice == null) continue;  // If the user cancels the input dialog, continue to the next iteration of the loop

    try {
        // Use a switch statement to execute the logic based on the user's choice
        switch (choice) {
            case "1": {
                // Handle account creation logic here
                break;  // Exit the current case block
            }
            case "2": {
                // Handle viewing account details logic here
```

```
                break;  // Exit the current case block
            }
            case "3": {
                // Handle deposit/withdraw logic here
                break;  // Exit the current case block
            }
            case "4": {
                // Handle fund transfer logic here
                break;  // Exit the current case block
            }
            case "5": {
                // Handle viewing transaction history logic here
                break;  // Exit the current case block
            }
            case "6": {
                // Handle report generation logic here
                break;  // Exit the current case block
            }
            case "7": {
                exit = true;  // Set the exit flag to true to break out
of the loop and exit the program
                JOptionPane.showMessageDialog(null, "Exiting
the system.");  // Show a message indicating the exit
                break;  // Exit the current case block
            }
            default: {

                JOptionPane.showMessageDialog(null, "Invalid
choice. Please try again.");
// Show an error message if the user enters an invalid choice
```

```
            }
        }
    } catch (Exception e) {
            JOptionPane.showMessageDialog(null, "Error: " +
e.getMessage());
// Catch any exceptions that occur during the menu handling
and show an error message
    }
}
```

**6. Graphical User Interface:**

```
JOptionPane.showInputDialog(menu);
 // Used to display input dialog for user selections

JOptionPane.showMessageDialog(null, "Account created
successfully!\n" + account);  // Display message

JOptionPane.showMessageDialog(null, "No transactions
found.");  // Display message
```

**7. Data and Time Management:**

```
String date = new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss").format(new Date());
```

*// Get the current date and time, formatted as "year-month-day hour:minute:second"*

**8. File Processing:**

```
try (FileWriter writer = new FileWriter("transactions.log", true)) {
// Open a FileWriter in append mode to write to the "transactions.log" file

    String timestamp = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss").format(new Date());
        writer.write(timestamp + " | " + this + " | " + transaction + "\n");
// Get the current timestamp in the format "yyyy-MM-dd HH:mm:ss"
// Write the timestamp, account details, and transaction details to the file, followed by a newline character

} catch (IOException e) {
 // Catch any IOExceptions that may occur during the file writing process

    JOptionPane.showMessageDialog(null, "Failed to log transaction: " + e.getMessage());
// If an error occurs while writing, show a message box with the error message
}
```

## Conceptual Description

Encapsulation – The system applied encapsulation during the designing of variables and methods within the Superclass. These methods could only be accessed by objects of the Superclass and its Sub-classes, hence protecting the info of it from being manipulated freely.

For example: Several variables like accountID, customerID, amount, etc being made private to prevent unauthorized manipulation of these variable's values

Inheritance – The usage of Sub-classes derived from the Superclass clearly shows the implementation of inheritance. This decision allowed is to reuse code from the Superclass to provide similar functions to both Sub-classes.

For example: The creation of Sub-classes SavingsAccount and CurrentAccount extending BankAccount

Polymorphism – The knowledge of polymorphism was applied using method overriding where the Sub-classes managed to implement the methods provided by the Superclass and were able to derive their respective values from these shared methods. This allowed us to save time and effort on the overall coding.

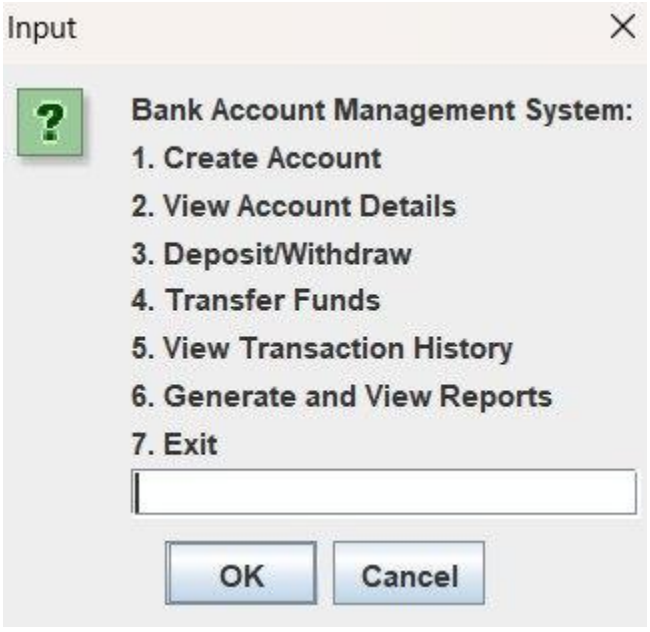
For example: Methods like setAccId() were overridden by Sub-classes to maximize functionality

Abstraction – To prevent the creation of unnecessary Superclass objects, we opted to make the Superclass an abstract class, hiding its functionality from the user but allowing the sub-classes to make good use of its abstract methods. This also helped to increase the safety of the code with the added benefit of making the overall code look a little neater.

For example: The Supeclass BankAccount was made abstract to prevent user from making an uneccessary object of it

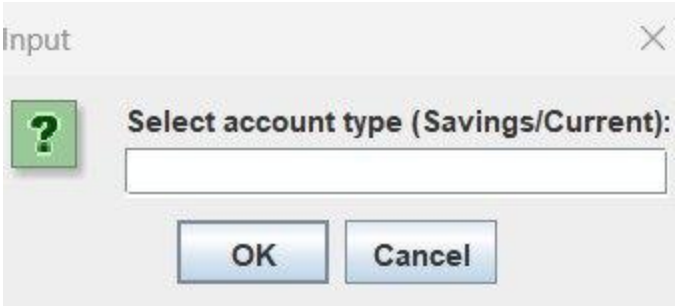
## Screenshots

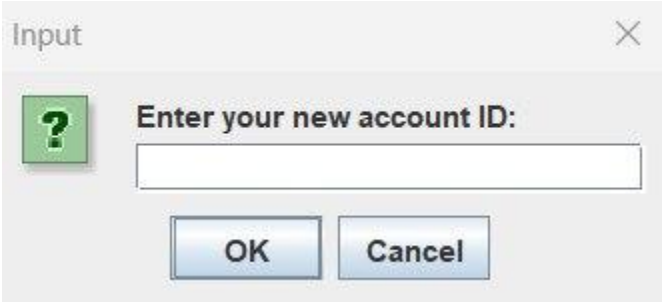
**Graphical User Interface(GUI)**

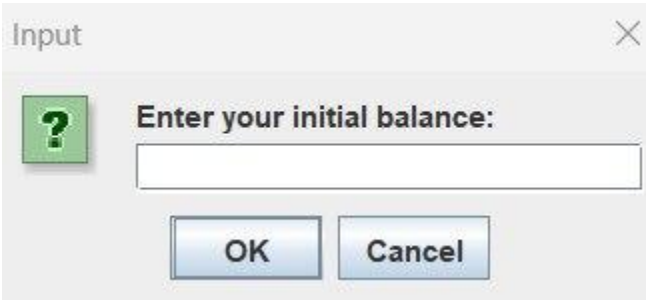



**Output Result**

**1.**

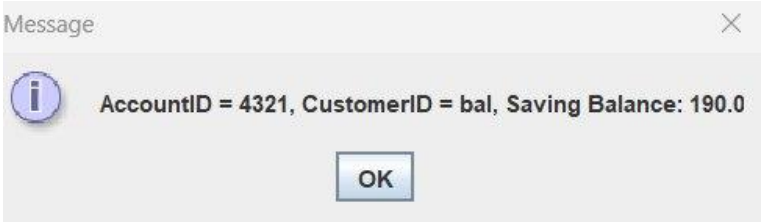

Input
Select account type (Savings/Current):
OK
Cancel


Input
Enter your new account ID:
OK
Cancel

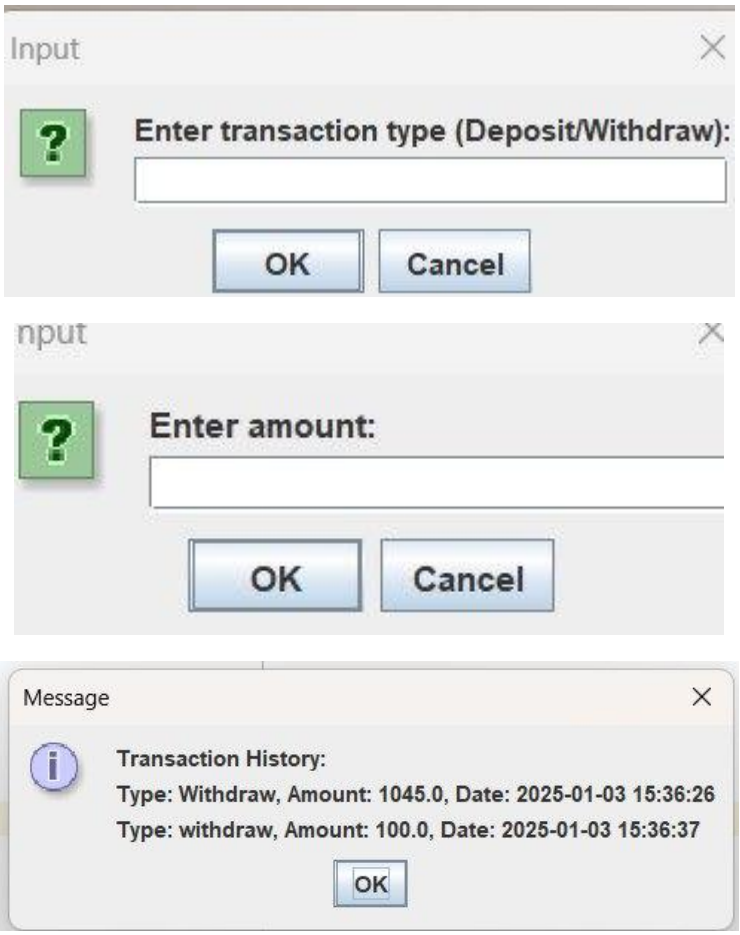

Input
Enter your initial balance:
OK
Cancel


Message
Account created successfully!
AccountID = 1567, CustomerID = irfan, Saving Balance: 10009.0
OK

**2.**


Message
AccountID = 4321, CustomerID = bal, Saving Balance: 190.0
OK

**3.**

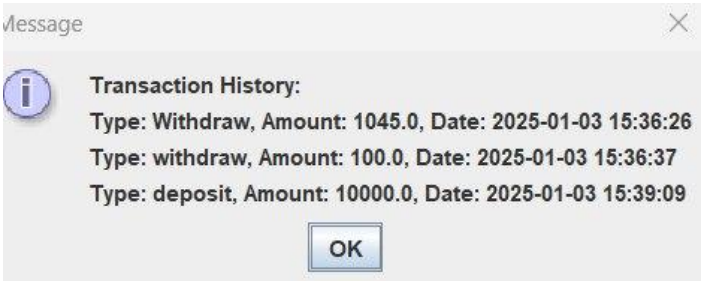

Input
Enter transaction type (Deposit/Withdraw):
OK
Cancel
nput
Enter amount:
OK
Cancel
Message
Transaction History:
Type: Withdraw, Amount: 1045.0, Date: 2025-01-03 15:36:26
Type: withdraw, Amount: 100.0, Date: 2025-01-03 15:36:37
OK

**4.**

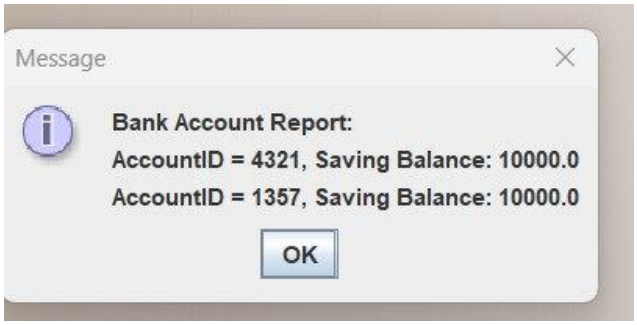

Input
Enter your source account ID:
OK
Cancel

**5.**



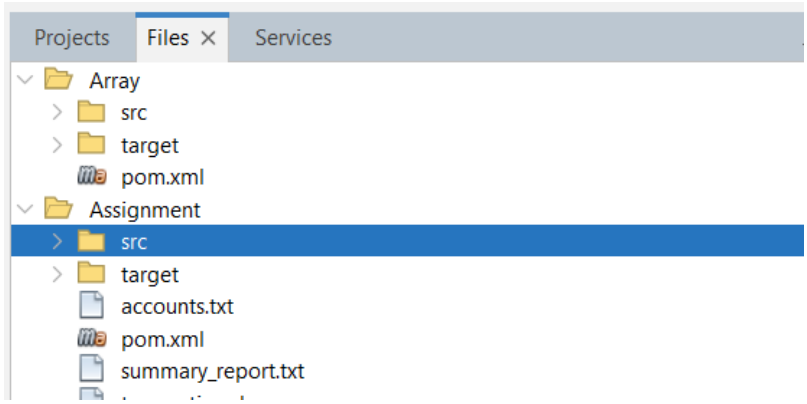

**6.**




**7.**



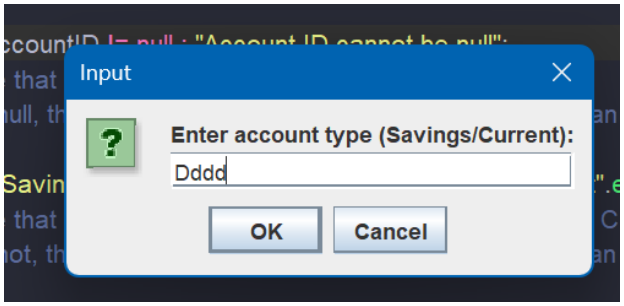

**File creation**




**Summary report creation**

```
AccountID = 123, CustomerID = 123, Created On: 2025-01-03 07:29:35
AccountID = fik, CustomerID = 123, Created On: 2025-01-03 07:29:35
AccountID = 1, CustomerID = 1, Created On: 2025-01-03 07:29:35
Type: Transfer In, Amount: 1000.0, Status: SUCCESS, Date: 2025-01-03 07:30:21
AccountID = 1, CustomerID = 1, Created On: 2025-01-03 07:29:50
AccountID = 2, CustomerID = 2, Created On: 2025-01-03 07:30:08
Type: Transfer Out, Amount: -1000.0, Status: SUCCESS, Date: 2025-01-03 07:30:21
```

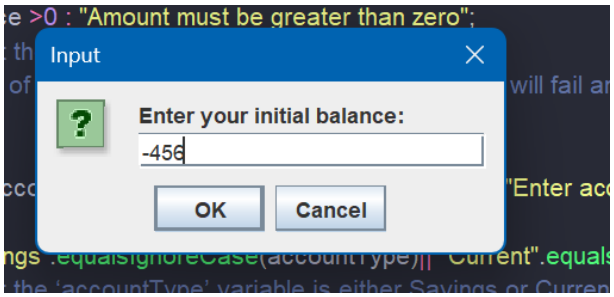
**Assertion validation (account type)**




```
--- compiler:3.13.0:compile (default-compile) @ OOP3 ---
Nothing to compile - all classes are up to date.

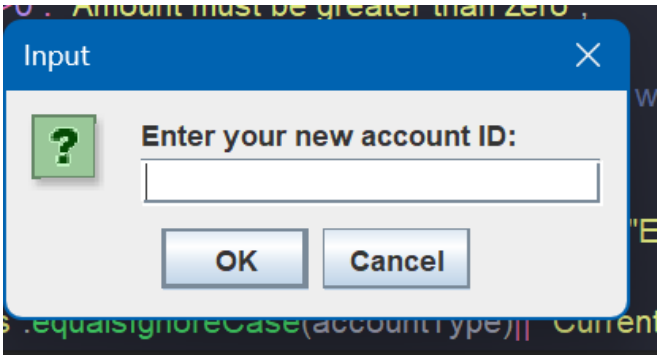
--- exec:3.1.0:exec (default-cli) @ OOP3 ---
Exception in thread "main" java.lang.AssertionError: Account type is invalid
        at com.mycompany.oop3.OOP3.createAccount(OOP3.java:134)
        at com.mycompany.oop3.OOP3.main(OOP3.java:111)
Command execution failed.
org.apache.commons.exec.ExecuteException: Process exited with an error: 1 (Exit value: 1)
```

**Assertion validation (Balance amount)**




```
--- exec:3.1.0:exec (default-cli) @ OOP3 ---
Exception in thread "main" java.lang.AssertionError: Amount must be greater than zero
        at com.mycompany.oop3.OOP3.createAccount(OOP3.java:132)
        at com.mycompany.oop3.OOP3.main(OOP3.java:111)
Command execution failed.
org.apache.commons.exec.ExecuteException: Process exited with an error: 1 (Exit value: 1)
    at org.apache.commons.exec.DefaultExecutor.executeInternal (DefaultExecutor.java:404)
    at org.apache.commons.exec.DefaultExecutor.execute (DefaultExecutor.java:166)
    at org.codehaus.mojo.exec.ExecMojo.executeCommandLine (ExecMojo.java:1000)
    at org.codehaus.mojo.exec.ExecMojo.executeCommandLine (ExecMojo.java:947)
```

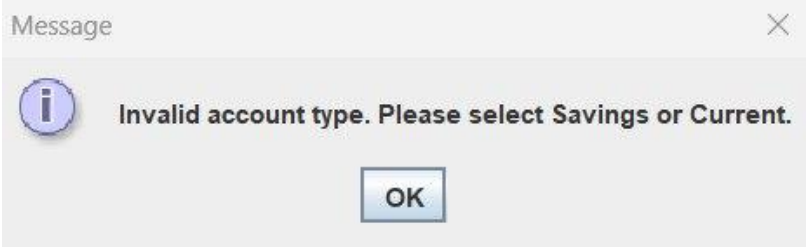
**Assertion validation (null account)**



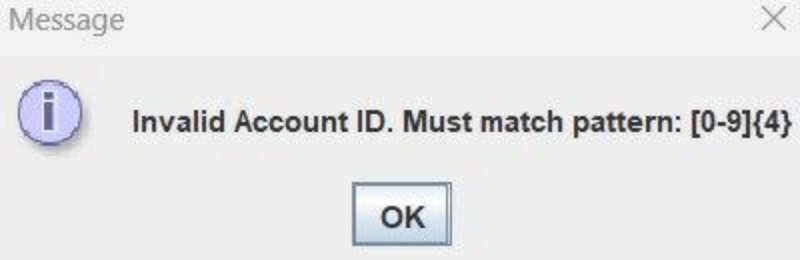

```
--- compiler:3.13.0:compile (default-compile) @ OOP3 ---
Nothing to compile - all classes are up to date.

--- exec:3.1.0:exec (default-cli) @ OOP3 ---
Exception in thread "main" java.lang.AssertionError: Account ID cannot be null
        at com.mycompany.oop3.OOP3.createAccount(OOP3.java:125)
        at com.mycompany.oop3.OOP3.main(OOP3.java:111)
Command execution failed.
org.apache.commons.exec.ExecuteException: Process exited with an error: 1 (Exit value: 1)
```

**Exception to check account type**




**Exception to check account number**




## Conclusion

We learnt the importance of implementing key features of Object-Oriented programming, those being Abstraction, Inheritance, Encapsulation and Polymorphism, into our coding processes.

We found that the implementation of these methods allowed us to provide better security to our code's variables and methods while still retaining all the necessary functions.

We also learnt that code reusability helps to make overall coding easier which gives us more time to further refine and validate our code's effectiveness, hence allowing us to create a more robust and reliable system.

We hope that our proposed design will be useful in solving problems such as the banking system and aim to implement the knowledge we learnt to further aid us in tackling more complex problems that may come in the future.